Islands of Reach: A Censorship-Resistant Federated Forum for National Internet Blackouts

Anonymous Author(s)

Abstract

A national internet shutdown does not arrive as one event. During the 2024 disruption in Bangladesh, people met throttling, then blocked platforms, then interference with TLS handshakes, and finally a five-day blackout. The four major circumvention designs we examine retain a path to infrastructure outside the censored region; a full transit cut removes that path. We present a public forum — communities, posts, threaded comments, votes and moderation, under pseudonymous identities — built so that the loss of international transit degrades its reach without ending it. Independent instances federate directly, and where even domestic peering is cut, a node holding transit on two ISPs bridges the resulting islands. Every mutation is a signed envelope that carries its own proof of authenticity and receives the same verification whatever transport delivered it, so trusting a peer raises how much it may send and never whether what it sends is valid. Content is valid the instant its author signs it, which removes withheld approval as a censorship mechanism; admission cost is charged at the origin, because an anti-abuse secret held by one node is meaningless at another; and spam is priced rather than policed by identity, because requiring identity to stop spam hands the adversary the lever it wants. We evaluate on a container testbed with independent datastores and separate address spaces, reporting multi-hop propagation, bridged crossing latency under a severed exchange, a measured denial-of-service surface, resident memory against a single-board-computer budget, and encoded size against a measured JSON-LD federation baseline. We also report an availability defect that only a deployed multi-peer testbed could expose: one unreachable peer stalled delivery to every reachable one, costing 407 s per crossing until we isolated and bounded it.

CCS Concepts

• **Networks** → **Network reliability**; • **Security and privacy** → *Distributed systems security*.

Keywords

censorship resistance, internet shutdowns, federation, pseudonymous publishing, network partitions, transparency logs, canonical encoding

1 Introduction

Internet shutdowns are usually described as on or off. They are not. The record of the July–August 2024 disruption in Bangladesh is a sequence of separate technical conditions [28]: selective throttling, blocking of individual platforms, interference with TLS handshakes, and then a national blackout lasting five days. Each condition breaks a different assumption, and a tool that handles one can fail completely at the next. How the blackout was produced matters for what can survive it: the broadband cut was executed upstream, through instructions issued to international terrestrial cable operators, the submarine cable company and IIG providers [28], so what was withdrawn was the country's transit rather than its internal networks. Shutdowns of this shape are not rare enough to treat as an edge case [6].

Figure 1: The resilience ladder. A different action cuts each rung, and a tool that survives one can fail entirely at the next. Circumvention systems require some remaining L0 path to infrastructure outside the censor; common federated social systems also assume global reach. We measure L0–L3.

What people lose in that sequence is not only messaging. It is the ability to publish to strangers: to say what is happening in a place, to be read by people who were not already in your contact list, and to have what you said remain checkable afterwards. That is a forum, and a forum is harder to keep alive under a shutdown than a chat application, because it needs durable content, a notion of community, moderation that is not itself a censorship lever, and identities that are recognisable without being attributable.

We name the conditions as rungs of a ladder. **L0** is unimpeded global reach. **L1** is a national partition: hosts inside the country can reach each other but not the outside. **L2** is an ISP island, where even domestic peering is severed and a network reaches only its own subscribers. **L3** is two or more such islands rejoined by an operator that holds transit on both. Figure 1 summarises the ladder. The system needs an IP path; what it does not need is an *unfiltered* one, or one that leaves the country.

Scope. We make no claim below L3. The repository also contains a separate identified emergency-communication plane, local WebRTC exchange, portable offline bundles and an optional Reticulum adapter. They are product features, not evidence for the result reported here. Off-grid mesh, Bluetooth relaying and long-range radio are a different problem with a different literature, and nothing in this paper depends on them. Tor access is discussed separately because it can help under selective blocking, but it still needs a reachable Tor path and therefore cannot establish the L1–L3 claim.

The gap we address is that the two families of systems built for exactly this adversary both stop at L1, and for the same reason. *Circumvention* systems — Tor's blocking-resistant design [15], domain fronting [18], decoy routing [36], Snowflake [8] — each work by reaching something the censor does not control, which is a host outside the censored region. Telex says so outright: governments that "disconnect entirely in times of crisis" are "outside our threat model" [36]. *Federated* social systems — ActivityPub [35],

Matrix [32], the AT Protocol [7] — assume L0 and define federation as delivery to a named remote host, and their reachability is more brittle than the architecture suggests: removing instances hosted in five autonomous systems from Mastodon's measured graph cuts its largest connected component from 92% of users to 46% [27]. A population under a national blackout needs the rungs below L1, and that is where neither family operates.

We should say at the outset that the *mechanisms* we use are not new. Addressing content by the hash of its canonical encoding and signing it at rest is Nostr [17] and Secure Scuttlebutt [31]. Gossiping tree heads to detect a rewritten log is Certificate Transparency [23]. Separating credential issuance from credential verification is Privacy Pass [13]. Preferring the narrowest working path while keeping alternates warm is ICE [22] and MPTCP [19]. What we contribute is the composition, the invariants that composition forces on us, and what it costs once measured.
This paper contributes:

- **Canonicalisation as an *admission* test, and the three-position design space it completes.** Prior systems either canonicalise inside the verifier (Matrix) or refuse to define a canonical form at all (Cosmos SDK). These are two of three positions. We take the third, and it buys a property we can state plainly: no canonicalisation function ever sits between an attacker and a verifier, so a relay that re-encodes a message cannot quietly *repair* a malformed object into a verifiable one (§3.2).
- **A conservative fork policy when no consistency proof can be fetched.** The CT gossip literature settles an ambiguous pair of tree heads with a consistency proof, and fetching one costs a round trip to the peer whose reachability is already in doubt. That makes it unavailable during a partition, which is when fork detection is most likely to fire. We show that the resulting false accusations are real and severe (one uplink flap wrongly blocked two of three honest peers), give a weaker local rule that avoids that false positive without a round trip, and state explicitly that it neither proves consistency nor identifies an honest branch (§4.4).
- **Two federation invariants, each stated with the deployed failure that produced it.** An identifier inside a federated payload has to be derivable by any node from the signed bytes, and admission cost has to be charged at the origin, because a node's local anti-abuse secrets mean nothing anywhere else (§4).
- **A partition testbed, a resilience ladder that makes such systems comparable, and the first measurements of this one:** propagation measured over hop count and queueing parameter, an island crossing under a severed exchange, resident memory against a single-board-computer budget, encoded size against a measured ActivityPub baseline, and a denial-of-service surface measured rather than asserted (§6).

2 Related Work

This venue spans networking and security broadly, so we introduce each concept instead of assuming it.

Censorship-resistant publishing. The problem is not new, and neither is our adversary. Publius [34] splits a document across servers so no single operator can remove it; Freenet [10] makes stored content unidentifiable to the node storing it; Tangler [33] entangles blocks so a server has reason to keep any particular one. What unites them is the threat they solve, *takedown of the hosts*, and the remedy they reach for, *geography*. Tangler names our attack exactly — "an attack with the same end result is to simply force the hosting servers off the network so that potential readers cannot contact the servers" — and answers it by replicating across "many different countries and judicial domains" [33]. That answer requires the reader to reach those countries. It resolves a takedown and does nothing for a blackout, and closing that gap is what this paper is for. The circumvention systems named in §1 share the same dependency and say so themselves [8, 15, 18, 36].

Federated public discussion. Netnews is the historical baseline: independent servers relay topic-grouped public articles, retry transfers and apply local moderation and acceptance policy [2]. Its base format does not authenticate authors. Nostr [17] and Secure Scuttlebutt [29, 31] instead use author-signed objects that readers verify without a server. SSB replicates among addressable peers; base Nostr has no relay-to-relay exchange, although a client can copy events and draft NIP-77 defines optional reconciliation [26]. None specifies our scope-aware multi-ISP availability policy.

Federation over IP.. ActivityPub [35] is the dominant federation protocol and makes neither object signatures nor HTTP Signatures normative; its authentication discussion sits in a non-normative appendix. Matrix [32] signs events over a Canonical JSON form, and has been analysed adversarially [1]. The AT Protocol [7] is the closest prior composition to ours — signed commits over a Merkle search tree with deterministic CBOR — but its verification depends on an external identity directory operated by a single company [4], which a shutdown removes. Decentralised moderation in these systems is itself a studied problem [3], and it shapes our decision to make moderation additive rather than gatekeeping.

Transparency logs. Certificate Transparency [23] established the pattern we reuse: an append-only Merkle log [25] whose signed tree heads are gossiped, so anyone comparing two heads can detect a participant that rewrote history. Comparing heads safely is itself well studied [12, 24], and every applicable resolution needs a consistency proof. §4.4 is about the window in which that proof cannot be fetched.

Anti-abuse without identity. Pricing admission rather than checking identity [16], blind signatures [9] and their modern deployment as Privacy Pass [13, 14] let a system charge an anonymous participant. We adopt this because the alternative is worse than inconvenient: the UN Special Rapporteur's finding is that identity requirements are themselves a restriction on expression [21], so an anti-abuse design that demands identity hands the adversary the lever it wants.

Path selection. ICE [22] and MPTCP [19] show that holding several candidate paths and continuously preferring among them beats failing over on demand. We apply the same reasoning to reachability *scope* rather than to addresses.

3 Design

3.1 One write path

The system exposes exactly one write endpoint. A post, a vote, a moderation action, a key revocation and a broadcast are all the same structure: an *envelope* carrying a domain string, an author key, a timestamp, a priority class, an opaque body and a signature. There is no per-feature write route. Adding a feature adds a registry row and a handler, not a branch anywhere in the ingress path.

The envelope's identifier comes from its own bytes:

$$id = \textsc{base32}\big(\textsc{sha-256}(canonical(fields_{1..12}))\big)$$

and the Ed25519 [5] signature covers those same bytes. Nothing in this identifier depends on which node computed it, which is what lets two independent instances agree on identity without coordinating.

The Forum registry covers communities, memberships, posts, threaded comments, votes, attachments, profiles, social actions, pseudonymous messages and signed moderation records. The same codebase has a separate Signal plane for identified channels, ordered emergency broadcasts, check-ins, crisis reports and encrypted messages. Plane-scoped certificates prevent a Forum pseudonym from authorising a Signal action. This paper evaluates the Forum plane; the Signal feature set is stated here to define the system boundary, not to enlarge the evaluation claim.

3.2 Canonical encoding as an admission gate

Matrix re-encodes parsed events as Canonical JSON and verifies that result [32]; Cosmos instead verifies received protobuf bytes and rejects a canonicalisation requirement as extra client complexity that restricts upgrades [11]. We combine a defined form with the latter's verification rule, but order them differently. Before verification, the decoder re-encodes the parsed message and rejects any mismatch. The signature is then checked against **the bytes as they arrived**, never against repaired bytes. Canonicalisation therefore decides what to admit, not what to verify against.

This prevents a relay from turning a malformed encoding into a valid-looking object by re-serialising it. Protobuf itself is not canonical [20], so our profile defines one accepted form and proves it across implementations (§6.2). The cost is explicit: unknown fields are not retained and compatible evolution needs a version bump.

3.3 The validation pipeline

Every envelope, from every transport, runs the same nineteen-step pipeline in the same order (Figure 2). Naming all nineteen matters because two of the orderings are load-bearing security properties rather than implementation taste, and neither is visible from a summary.

Steps 1–12 write nothing to the database. Everything that can be decided from the bytes and the local index alone is decided first: size on the raw bytes before parsing, canonical decode, registry lookup, clock bounds, the signature, the author's certificate, the dedupe index and the replay check. Only at step 13 does the node touch storage. A flood of invalid envelopes therefore costs CPU and nothing else, which we measure in §6.6.

Steps 16 and 17 commit together. The projection and the Merkle append occur in one transaction, because an envelope visible in a projection but missing from the log is exactly the transparency failure the log exists to prevent. This forces a deployment constraint — the storage engine must support multi-document transactions — which we state rather than hide.

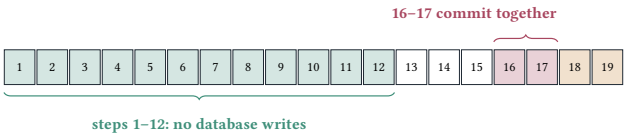

Figure 2: The nineteen steps, in order. *1* size, on raw bytes before parsing · *2* parse, canonical decode with unknown fields rejected · *3* version · *4* domain · *5* plane · *6* algorithm policy · *7* priority · *8* clock · *9* signature · *10* certificate · *11* dedupe · *12* replay · *13* anti-abuse · *14* authorise · *15* body validate · *16* apply · *17* witness · *18* receipt · *19* fanout. An inbound federated envelope re-runs all of them except 13.

A federated envelope does not get weaker verification. Every applicable step re-runs at the receiver. Step 13 is the sole exception, for a precise reason given in §4: the cost was charged at the origin and its proof is keyed to that node, so what is skipped is a payment and not a check. Each step is a pure function that can be tested on its own.

3.4 The audit-log workflow

Modification, deletion and omission are different attacks. A signature detects modification but not whether a server accepted an object. Figure 3 follows one public write. The client queues the final signed request before sending it. After admission, steps 16–17 commit the projection and Merkle leaf together; step 18 returns a node-signed receipt with the server and content identities, leaf position, signed tree head and inclusion proof. The client binds that receipt to the *exact UTF-8 request body*, verifies the resulting audit certificate and saves it locally first.

The client forwards the certificate to each configured independent audit-log server (ALS). Each ALS decodes the original envelope, enforces canonical encoding, recomputes its identifiers, verifies the receipt, tree-head signatures and inclusion proof, then appends it to a JSONL hash chain. Exact retries are idempotent; conflicting certificates for one $(server, content)$ pair are rejected. Failed deliveries remain pending, and provenance failures enter a separate hash-chained issue stream.

Later, the client presents the certificate to the *issuing node*'s `/status` endpoint, which reports online, hidden, deleted or wrong-server status. The ALS preserves the earlier acknowledgement; it does not claim current node state. Public certificates may be copied externally, but direct-message certificates remain local to avoid exposing a timestamped social graph.

This is evidence, not immutable storage. It proves prior acknowledgement, but cannot prove acceptance without a receipt, compel service, or recover a body after every copy is destroyed. An ALS can also withhold its copy; replication helps only when clients and auditors are independent.

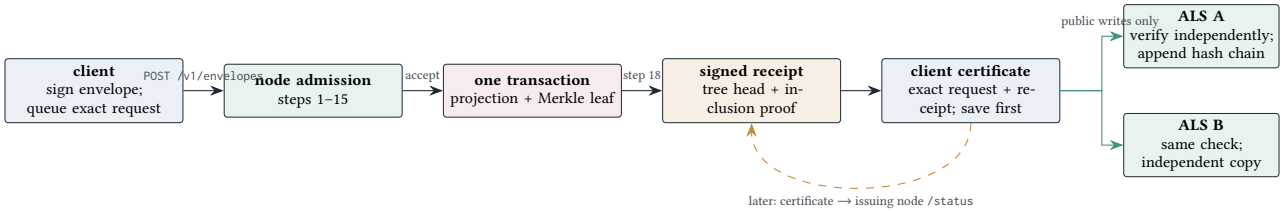

Figure 3: One write path and the ALS workflow. The client keeps the certificate before forwarding it, so audit delivery can resume after an outage. An ALS verifies a node's prior acknowledgement and retains it independently; only the issuing node reports whether the content is currently online, hidden or missing.

3.5 Additive labels as an open taxonomy

Moderation here is already additive: a `Label` is a signed opinion about content that is already published, never a precondition for publishing it, and removal leaves a tombstone rather than an absence. What is easy to miss is that this already holds at the protocol level, not only as policy. A label's category field is a free string rather than a closed enumeration, its model or moderator identifier is likewise free text, and an *unscoped* label needs no community's permission to publish at all — a third-party fact-checker or community moderation team writes one exactly as the local instance would, over the same one write path (§3). No vocabulary and no publishing key is privileged by the protocol.

We take this as evidence that the same mechanism generalises past a single safety verdict. A node behind a partition benefits from surfacing *who is asking for help* at least as much as *what should be hidden*, so we specify — but, in the sense that §1 states the Signal plane to define the system boundary rather than to enlarge the evaluation claim, do not here evaluate — an extension from one closed verdict to open `(taxonomy, label, score)` triples covering sentiment, topical intent and language. A taxonomy this node has never seen before still canonicalises, signs, verifies and federates, because it is carried as data rather than adopted as protocol; publishing it costs a registry row and a handler, and the admission path in Figure 2 gains no new step, because an assertion is an ordinary envelope through the ordinary path (Figure 4). What changes for a client is only which taxonomies a given node advertises, over a manifest it is free to ignore.

Two consequences follow directly from reusing rather than extending the write path, and neither is specific to sentiment or intent. First, whatever classifier an operator runs cannot gate a post, because nothing between §3.2 and §3.4 consults one; the worst it can do is answer late or not at all, which is the same failure mode §2 already accepts for any labeller. Second, no one classifier's output is structurally load-bearing: the taxonomy it speaks is metadata a reader may act on, not a gate a publisher must pass.

4 Federating Across a Partition

Federation is server to server only. Clients do not speak it, because operators choose their peers and clients do not, and because the protocol's connection profile degrades badly on lossy mobile links.

The ISP subsystem is separate from the nineteen admission steps. Its 22 implementation tasks cover four groups: scoped endpoints, probes, source binding, ranking, backoff and metrics; durable

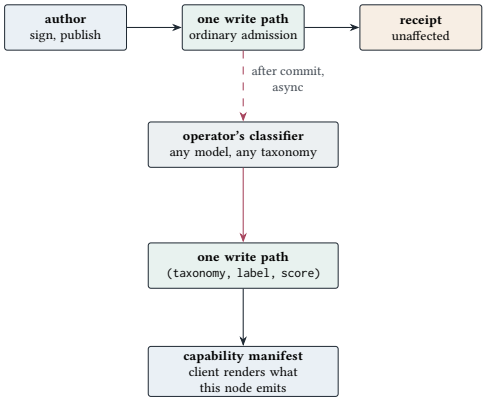

Figure 4: Publishing is unaffected by classification: the top row is the ordinary write path and returns its receipt whether or not any classifier ever runs. An accepted post's content ID is handed to an operator-chosen classifier only afterward and off the critical path; its opinion re-enters through the same write path as an ordinary envelope, carrying an open taxonomy rather than a fixed vocabulary, and a capability manifest tells a client which taxonomies this particular node speaks. This design is specified, not evaluated.

peer and seed discovery plus local/manual discovery and NAT traversal; opt-in bridge relay, reserved quotas, uplink failover, reannouncement and backfill; and client/operator visibility. Bridging itself is not a twentieth verifier: it is the policy attached to step 19, after the received envelope has passed the same applicable admission checks as a local one.

4.1 Four rules that carry the weight

Peer bytes are never re-encoded. The transport uses a passthrough codec. The decoder establishes canonicality by re-encoding and comparing (§3.2), so a round trip through a generated codec inside the network adapter would quietly repair an untrusted peer's non-canonical envelope into one that validates. That is the exact hazard the admission gate exists to stop, arriving over the network and going around it.

Trust bounds volume, never validity. Inbound envelopes re-run every applicable step. A trusted peer's forged envelope is rejected exactly as a stranger's is. Trust sets the quota and the priority classes a peer may push, and nothing else.

Deduplication is a unique index, not a read-then-write. The pair (*content_id*, *direction*) is a uniqueness constraint in the database. A read-then-write check is unsound under the concurrency federation produces, where an overlapping stream and a backfill can deliver the same envelope twice at once.

An envelope is never relayed to its sender. The pipeline carries the origin peer and excludes it from fanout. Content dedupe makes such a loop terminate, but it does not make it free: without this rule, every envelope pays a round trip before being thrown away.

4.2 Two invariants, and the failures that produced them

An identifier in a federated payload has to be derivable from the signed bytes. An earlier version derived a community identifier from the projecting node's own key. With one node that is indistinguishable from correct, because the projecting node *is* the origin node. With two, the same signed envelope produced a different community identifier on each instance, and the projections diverged for good. The rule that replaced it is a question asked before any identifier is minted: *would a different node, given only these bytes, compute the same value?*

Admission cost is charged at the origin. Proof-of-work, credit balances, blind credentials and epoch nullifiers are each deliberately keyed to one node. A proof minted at the origin is therefore not merely unverifiable elsewhere, it is meaningless elsewhere. Charging again on receipt would make every anti-abuse-gated domain impossible to federate. Cost is paid once, at the origin, and the receiver's protection is a per-peer, per-class quota with an explicit backpressure hint. Verification is untouched: what is skipped is a payment, not a check.

One consequence is that a peer over quota is not disconnected. It would reconnect immediately, pay the handshake again and arrive in the same state, so refusing costs more than accepting. The hint lets a well-behaved peer slow itself down, and a peer that ignores the hint is demoted.

4.3 Federating from a node nothing can dial

In ActivityPub, Matrix and the AT Protocol, server-to-server delivery is a *push to an address the receiver publishes*, so a node with no inbound reachability cannot receive at all unless a third party holds an address for it. In the deployments this paper is about, that is the common case rather than the corner case: a community node is a machine in somebody's home, behind carrier-grade NAT. We make reachability an asymmetry instead of a requirement, so a node may advertise *no* endpoints and still federate both ways:

- **outbound** is the durable outbox draining over a connection the node opened;
- **inbound** is a resumable backfill the node *requests*, also over a connection it opened, so nothing is pushed to it and nothing needs to reach it.

We assert the property instead of arguing it. An unreachable node is introduced to a peer; the peer records it with an empty endpoint list and can never dial it; the test then shows a post published on the unreachable node arriving at the peer, and a post published on the peer arriving at the unreachable node. Neither direction is started by the reachable side.

Tor access has a different role. The deployment scripts can publish the node's local HTTP port as a Tor v3 onion service, and the mobile client can select an embedded Tor transport when given an onion address. This helps when direct addresses are filtered or an operator cannot accept ordinary inbound connections. It does not bridge the national partition studied here: Tor still requires a path into the Tor network [15]. We therefore treat it as a complementary L0 access path, not as evidence that an L1 or L2 component can reach another component.

4.4 Fork detection when a consistency proof is unavailable

A transparency log lets a participant detect a peer that rewrote history. Tree heads are signed and gossiped, and a head smaller than one already seen suggests entries were removed. This is the Certificate Transparency construction [23], and we adopt it unchanged.

What the gossip literature already settles. Comparing tree heads pairwise is subtle, and the CT gossip protocols address it directly. An STH-only gossip design observes that when one tree provably *extends* another, the smaller head can be discarded without lowering the chance of detecting an attack [12]. The resolution in that literature is a **consistency proof** between the two heads. CT v2 deliberately leaves gossip as active research [24]; obtaining the proof or consulting an auditor still costs a round trip that a partition may remove.

What a partition changes. (In the CT literature a "partitioning attack" means a log equivocating between clients; we mean a network partition throughout.) A consistency proof has to be fetched from the log. That is a round trip to the very peer whose reachability is in question, and in our setting that peer may sit behind the cut being routed around, which is precisely when fork detection is most likely to fire. The standard resolution is available exactly when it is least needed.

The failure this produces is not a missed detection but a false one, which is worse. In our node, four independent code paths observe a peer's head: the outbound handshake, the inbound handshake, a head carried on a peer record, and the gossip round. Each samples at a different instant and nothing sequences their arrival, so *gossip records size 8, then a reconnect delivers the size-6 head it captured moments earlier* is ordinary concurrency. Compared pairwise with no proof, it reads as a tree that shrank. A fork is the strongest signal the system has, so the peer is demoted to a state only an operator can lift. The node thus partitions itself from honest peers, during a partition, on evidence it manufactured. In our deployed four-node testbed, one uplink flap did this to **two of three peers at once**, both with the same manufactured reason (*tree shrank from 8 to 6*), and neither log had been rewritten.

A rule that costs no round trip. Signed tree heads already carry a signed timestamp, so the readings can be ordered without asking anyone anything. Let (t, n) be the timestamp and size of an incoming head and (t_0, n_0) those of the recorded one:

(1) if $t < t_0$ the reading is stale: return UNKNOWN, and **do not record it**;
(2) otherwise if $n < n_0$ the tree regressed: report FORKED;
(3) otherwise record (t, n) and report OK.

This does not replace a consistency proof. It is strictly weaker, and a node that can still reach the peer should fetch one. What it does is remove the false accusations in the window where no proof can be obtained, which is the window that matters here. A malicious log controls its timestamps: ordering heads is a local tie-break, not proof of append-only consistency or of which branch is honest.

The second half of clause 1 is the easy part to get wrong. The natural implementation discards the stale reading by overwriting the record with it, or by recording it and comparing next time. Both bring the bug back one step later: writing (t, n) back rolls the ledger to size 6, so a later honest observation is compared against a value that should never have been stored. The stale reading has to leave no trace.

Genuine rollback still trips, and we assert both directions. An attacker now has to present a head that is both newer *and* smaller, which still trips clause 2. The regression test asserts that a smaller-and-older head leaves trust intact and leaves the recorded size at 8, *and* that a smaller head with an equal-or-newer timestamp still reports FORKED. Without the second assertion, the guard would be indistinguishable from deleting the check.

4.5 Convergence under reordering

Two nodes may receive the same set of signed envelopes in different orders, so any projection that folds them in arrival order will diverge. We found this in a vote handler that merged by arrival: two nodes receiving the same two signed votes in opposite orders disagreed permanently on both the stored value and the derived score. The fix is a total order built only from fields inside the signed envelope, here (*created_at_ms*, *content_id*), so every node folds identically whatever the delivery order was. We assert the property over *every permutation* of a conflicting set, not over one arrival order, because a test that fixes the order cannot tell an order-independent fold from a lucky one.

4.6 Bridging two islands

At L3 two ISP islands are severed from each other, but some node holds transit on both — a university, a small ISP, an office with two providers. That node is not a special server. It is an ordinary federated node with two uplinks and a relay policy, which matters because a design that needed purpose-built infrastructure would need it to exist *before* the shutdown.

Source binding. Each outbound federation connection is bound to the source address of the uplink that serves the peer's island. Getting this wrong is invisible in a single-homed test and fatal in deployment: the connection leaves on the default route, arrives at the far island from an address that island cannot return to, and the crossing silently never completes. It is also the one property no in-process harness can assert, since a test can only check which address the selector *chose*. We read `/proc/net/tcp` inside the running container to confirm which address sockets actually left from.

A bridge verifies; it does not repeat. A received envelope runs the full verification path before step 19 may relay it onward; only the origin-priced payment at step 13 is not charged again. The bridge is the one node in the topology positioned to tamper with everything crossing between two populations, so it is given no authority it does not need: it cannot alter an envelope without invalidating a signature, and it cannot admit one that fails validation. Bridging additionally requires TRUSTED standing with at least one peer on each side, so an unknown node cannot volunteer itself as the chokepoint between two islands. It also never relays back to the uplink an envelope arrived on: content dedupe would terminate that loop, but only after paying a crossing, which is the scarcest resource in the topology.

Accounting is per uplink pair, not per peer. An island with three nodes is charged once for one link's traffic rather than three times. Charging per peer would make an island's cost depend on how many volunteers happened to stand up nodes there, which is precisely the quantity a shutdown makes unpredictable. Within a pair, bulk traffic draws on a bucket capped at half the pair's grant, so classes 0–2 — broadcast, direct message, check-in — retain reserved capacity that a forum backlog cannot consume.

Failover. An uplink state change re-evaluates every active peer path within 30 s. Nothing queued is lost, because the outbound queue is durable and uplink-agnostic — only the path changes. After a switch, affected peers are re-announced with the new endpoint set and a resumable backfill closes whatever gap the transition opened. An operator override can force an uplink up or down for conditions the probes cannot detect, which exists because the probe sees reachability and the operator sees the news.

5 Adversary and Scope

We ground the adversary in the measured event. It controls routing inside its own autonomous system and can withdraw or null-route prefixes; inspect transit it carries and block by SNI, IP or DNS; throttle selectively, including to the point where a protocol times out without failing cleanly; partition the country from the outside or one ISP from another; observe the volume and timing of federation traffic on links it controls; and run its own nodes as a legitimate peer, so it is a participant and not only an observer. We assume it cannot break Ed25519 or SHA-256, cannot extract a signing key from an uncompromised device, is not a *global* passive observer, and does not control a majority of the peer set. Those four are assumptions, not results.

One rule carries most of §4 and is worth stating on its own: **peer trust bounds volume, never validity**. Operators choose their peers, admission is trust-on-first-use at PROBATION with vouch-based promotion, and no admin allowlist exists — during a shutdown, volunteers stand up relay nodes and cannot wait for manual approval. What trust buys a peer is quota and traffic class. It never buys a weaker check.

Table 1: What the design resists, the mechanism, and the boundary of that claim.

Attack	The property that resists it
A server modifies content	Author signatures, content-derived identifiers and client verification detect any changed admitted byte; they do not prove freshness or completeness.
A server deletes, omits or moderates content	Acknowledged content leaves a client-portable receipt and audit certificate. Moderation is a signed additive opinion and removal leaves a tombstone. No mechanism proves an unacknowledged submission was accepted or forces a local read API to show it.
A relay tampers with content in flight	The signature covers the bytes *as they arrived*; no intermediary re-encodes them; a non-canonical encoding is rejected rather than repaired (§3.2).
A peer rewrites its own history	Signed tree heads gossiped between peers, with fork detection that stays safe when no consistency proof can be fetched (§4.4).
An invalid flood amplifies into database load	Pipeline steps 1–12 perform no writes. Measured: 730 req/s absorbed, zero writes.
A partition cuts reach	Continuous preference for the narrowest working scope, plus a multi-homed bridge (§4.6). This requires surviving domestic IP paths, local nodes, and a bridge wherever ISP islands cannot route directly.
Direct access to a node is blocked	A Tor v3 onion service, manual or QR-supplied node addresses, and an optional trusted reverse tunnel provide alternate access paths. Tor still needs a reachable Tor network; a tunnel provider observes the request it carries; neither mechanism prevents traffic correlation.
One peer frames another to get it silenced	A check that can BLOCK must first verify the claim belongs to the peer it names and is current. An older reading of remote state is not evidence.
Spam and Sybil flooding	Cost, not identity: memory-hard proof of work, credits, blind credentials and epoch nullifiers, all of which work without civil identity. This bounds rate, not identities, coordination or unequal access to compute.
Deanonymisation	Forum accounts require no email, password or civil identity, use public-key pseudonyms, and the node does not persist an IP-to-key map. This is not network anonymity: timing, traffic, writing style, key reuse and attachments remain linkable.

Table 1 separates evidence from availability: signatures catch an edit, receipts prove prior acknowledgement, replication may preserve a body, and none of the three proves that an operator showed every valid object. The sociotechnical threats remain sharper. A transparent moderation decision can still be abusive; an instance can defederate; a dominant client, seed list or label provider can become a de facto censor; and brigading, disinformation, illegal content, moderator burnout and operator legal pressure are governance failures rather than signature failures. No labeller or classifier holds protocol privilege over another (§3.5), which narrows this risk but does not close it: a dominant *client* can still decide which labellers a reader ever sees. Replication can also preserve personal or dangerous material that a victim needs removed. The design makes censorship attributable where evidence survives; it does not make moderation fair or global erasure possible.

The exclusions matter as much, and we state them at full strength because a frank limit is worth more than a soft one. There is **no traffic-analysis resistance**: no cover traffic, no mixing, and an observer on both islands can correlate flows. There is **no transport metadata protection**, so an ISP sees that a node federates, roughly with whom and roughly how much. **Endpoint compromise** is out of scope beyond revocation, and revocation only helps once it reaches peers, which is the thing the adversary is attacking. Fork detection **assumes one honest observer** gossips a conflicting head. There is **no formal unlinkability proof** between the pseudonymous and identified identity planes; that separation is architectural. Sybil resistance is **deliberately not derived from identity**, so it bounds the rate of abuse rather than the number of identities.

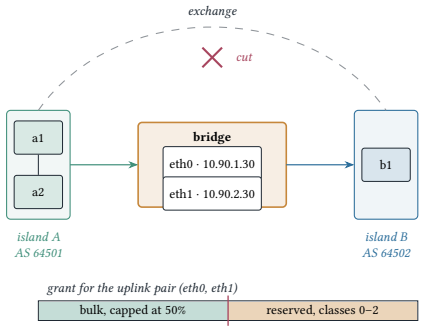

Figure 5: The partition testbed, which is also the bridging mechanism. The islands occupy separate address spaces on networks declared internal, so once the exchange is cut no route exists between them except through the bridge. The bridge binds each outbound connection to the matching uplink's source address, which we confirm by reading `/proc/net/tcp` inside the running container. Quota is accounted per uplink *pair*, and bulk draws on a bucket capped at half the grant so a forum backlog cannot starve an emergency broadcast.

And with every link cut and no path at all, the system stores and forwards; it does not deliver.

Three further residuals follow from the availability design. First contact is an **eclipse surface**: TOFU, retained seeds and vouching do not prove that a discovered peer set is diverse. A single bridge is a **coercible chokepoint**, even though it cannot forge content. Finally, the no-write prefix limits invalid-input amplification but does not prevent CPU, socket, bandwidth, queue or valid-work exhaustion. We have not measured redundant independently operated bridges, directory poisoning, clock drift, certificate expiry, slow peers or valid-envelope floods.

Each exclusion could be bought, and each would be bought with availability — the property the system exists to preserve — or with the ability to run on a single-board computer over a shared, intermittent link. Where a defence and reach conflict we choose reach, and say so.

6 Evaluation

6.1 Testbed

The measurements come from container testbeds on one host. The partition testbed runs four nodes across three networks, two of them declared internal so that no route exists between them except through the bridging node (Figure 5). Each island has its own database and cache, and the islands occupy separate address spaces. The scale testbed generates an N-node federation chain, $2 \le N \le 16$, in which node i is reachable from node 0 by exactly one path of exactly i hops.

Measurement boundary. Host contention confounded a drain-interval sweep, so we report one clean configuration rather than a parameter effect we could not isolate. Propagation uses the storage timestamps written with each projection. Container clocks are

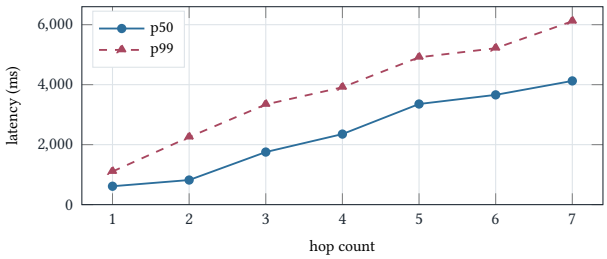

Figure 6: Propagation latency by hop count. Eight-node chain, 200 paced samples, zero loss: all 200 envelopes reached hop 7. Growth is linear in hop count, and the outbox drain interval sets the slope, not the protocol.

shared; a field deployment would need clock discipline and an error bound.

6.2 Cross-implementation agreement

The admission gate of §3.2 only works if the accepted encoding is genuinely agreed across implementations. Otherwise it turns into a compatibility bug that rejects honest peers. Three independent implementations of the canonical encoder, in TypeScript, Rust and Python, each written against the specification rather than ported from one another, agree byte for byte on all 16 conformance vectors, covering field omission, NFC normalisation, domain separation and plane separation. This is what makes strictness affordable, and it is our answer to the natural objection that two implementations will eventually disagree.

6.3 Correctness under partition

Across independent datastores, both nodes project the same post and origin-derived community identifier. Rebuilding every collection from the envelope log reproduces it byte for byte. The client also rejects a wrong receipt key, a tree head that omits the content and a content identifier copied from another envelope. The federation suite covers 31 assertions across admission, quota, streaming, backfill, fork detection and directory exchange.

6.4 Multi-hop propagation

Figure 6 reports an eight-node chain, 200 paced samples, with a 1000 ms outbox drain interval. All 200 envelopes reached hop 7, and the seven-hop median is 4125 ms against a p99 of 6115 ms.

Per-hop cost is set by the drain interval, not by the protocol. Predicting $drain/2 + fixed$ and measuring at two operating points isolates the constant. At a 1000 ms drain the seven-hop median implies 589 ms per hop and a fixed residual of 89 ms; at 250 ms it implies 196 ms per hop and a residual of 71 ms. The residual holds at **71–89 ms across a fourfold change in the queueing parameter**, and it is the protocol's real per-hop cost: verify, project, append to the witness log, re-enqueue. Everything above it is a scheduling choice an operator makes for battery and bandwidth reasons.

We report the offered-load regime with every figure, because it dominates the result. Publishing the same 200 envelopes as fast as the origin would accept them produced a first-hop median of 10.0 s and a 95th percentile of 173 s, while the *marginal* cost of

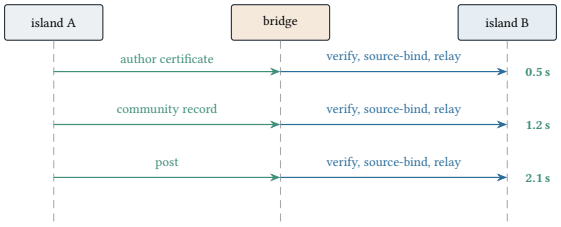

Figure 7: The measured dependency chain after the exchange is cut. The far island can render the post only after its author certificate and community record arrive first.

hops 2–7 stayed at 0.4–1.1 s. That shape is queueing and contention on a shared database, not propagation. Both are correct measurements of different questions, and averaging them would describe neither.

6.5 Bridged crossing

Crossing an ISP boundary is not one envelope. A newly published post that a receiving island can render depends on the author's certificate and on the community record, so three envelopes have to cross and project in dependency order (Figure 7).

The cut-path arrivals are 0.5, 1.2 and 2.1 s; the healthy-path figures are 1.5, 1.5 and 1.9 s. Once traffic uses the bridge, the cut does not meaningfully slow this crossing. The container gate also passes source binding, directional accounting, reserved capacity and uplink failover.

Inside the bridge, `/proc/net/tcp` confirms sockets leave from both configured source addresses, an assertion an in-process harness cannot make.

The first deployed run took 407.6 s: a serial, deadline-free drain blocked on one blackholed peer, so the bridge behind it was never attempted. Concurrent per-peer drains with deadlines restored the 2.1 s crossing. Multi-homing is defeated if a node inherits its worst peer's availability.

6.6 An invalid-envelope flood does not become write load

We flooded a node with well-formed, unique envelopes whose signed region had one mutated byte. Unlike random bytes, these reach the costly checks and cannot be removed by deduplication.

Against a single node, 3000 such envelopes at concurrency 16 were absorbed at **730 req/s**, and the node's database recorded **zero writes**. The envelope log was unchanged, and a collection-level profiler logged no insert, update or delete while still capturing read operations over the same window, which confirms it was live. Resident memory fell from 86 MiB to 61 MiB as the garbage collector caught up, and CPU returned to idle immediately.

Only 9% reached signature verification; **90% was shed by the per-peer rate limiter**. The no-write prefix is therefore the second line of defence, not the source of all measured throughput.

A defect surfaced. Eight requests initially returned HTTP 500 because metric labelling decoded untrusted bytes outside the pipeline and emitted stack traces. The label now falls back to a constant,

Table 2: Encoded size, ours against 80 real ActivityPub activities. Overhead leaves out authored text, which is a confound: our fixture carries a 14-character headline, while the sampled posts carry 30–524 B.

	p50	min	max
ActivityPub delivered (B)	4216	2786	6909
of which authored text	205	30	524
of which JSON-LD context	927	927	927
ActivityPub overhead (B)	**4015**	2756	6484
ActivityPub, gzipped (B)	1074	786	1400
Ours, check-in / post / broadcast (B)			155 / 220 / 243
Ours, overhead (B)			**155 / 206 / 177**

four adversarial encodings have regression tests, and rejection rose from 440 to 730 req/s.

6.7 Read path and footprint

The keep-alive read path peaks at 750 req/s (concurrency 8). An idle node uses 62 MiB RSS and 233 MiB under sustained crossing; node, database and cache together use roughly 384 MiB of a 512 MB target. Storage is the binding constraint on small hardware.

6.8 Encoded size against a measured ActivityPub baseline

Encoded envelopes including a 64-byte signature are 155 B for a check-in, 220 B for a forum post and 243 B for a Bangla broadcast. We compare them with 80 real `Create`/`Note` activities from two stock ActivityPub instances, re-attaching each host's JSON-LD context as remote delivery does.

After subtracting authored text, ActivityPub's overhead is **19.5×–25.9×** ours; its 927 B context alone is 4.2 times our signed post. Compression narrows this to **4.4×–6.9×**: signatures do not compress and gzip makes our envelopes larger. We omit HTTP framing and ActivityPub authentication, and count one activity rather than follower fanout. JSON-LD also buys extensibility our fixed schema does not; the table measures link bytes only.

7 Comparison with Related Systems

No quantitative baseline exists for "keeps a public forum readable across an ISP partition", so we compare capabilities. Table 3 scores the signed-content publishing systems that are genuine peers — every one of them has a checkable specification — on the ladder and on the integrity properties this design turns on. We checked every cell against the system's own specification.

Secure Scuttlebutt is stronger below L1: local broadcast discovery needs no configured infrastructure, whereas our lowest measured rung needs a multi-ISP operator. Nostr obtains no-inbound participation from its client-pull architecture. Server-to-server exchange itself is common, not novel.

ActivityPub scores ⊙ on tombstoning because Activity Streams defines the type, but does not require it to remain publicly retrievable [30]. AT Protocol scores ⊙ on offline verification: a cached DID key verifies an object, while a current key binding needs DID resolution [4, 7].

Table 3: Capability comparison. • provided and specified, ⊙ partial, optional or non-normative, ○ not provided. NF indicates the specification is silent. *No inbound*: a node with no reachable address still participates in both directions. *Svr*: the specification defines a server-to-server exchange at all. ActivityPub's partial score reflects measured hosting concentration and graph-removal analysis, not a protocol partition test [27].

	L0–L3 span	no inbound	signed at rest	verify w/o srv	no re-encode	log gossip	tombstone	svr ↔srv
ActivityPub	⊙	○	⊙	○	○	○	⊙	•
Netnews	⊙	○	○	○	NF	○	⊙	•
Matrix	⊙	○	•	•	○	○	•	•
AT Protocol	○	○	•	⊙	•	○	○	•
Nostr	⊙	•	•	•	⊙	○	○	⊙
Secure Scuttlebutt	⊙	•	•	•	NF	○	○	•
This work	•	•	•	•	•	•	•	•

No column here is novel on its own. What no prior system holds is the *conjunction* of server-to-server reach, participation without inbound reachability, and gossiped log heads, while spanning L0–L3 — and what this paper reports is the engineering cost of that combination rather than its novelty.

8 Limitations

One-host containers are a confound. Clocks are aligned, networks are virtual and nodes share storage. A repeated configuration changed first-hop median fivefold, so chain length and queueing remain unswept and there is no wide-area or lossy-link measurement.

A severed virtual bridge is not a field trial. Route removal reproduces lost reachability, not DPI, selective throttling or coercion. The partition result is a lower bound on mechanism cost.

Unlinkability is architectural, not proven. The separation between identity planes has no formal argument and no anonymity-set analysis, and we make no claim of resistance to a global passive observer.

One operator configured every node. Probation, vouching, demotion and governance remain unobserved between strangers.

Complementary transports and the Signal plane are not evaluated here. The implementation includes Tor client access, local WebRTC exchange, offline bundles, a separate identified Signal plane and an optional Reticulum sidecar. None contributes to the L0–L3 measurements. We have not measured Tor bootstrap or blocking resistance, local peer exchange, radio delivery, encrypted messaging or crisis-report usability under interference.

The open-taxonomy labelling extension (§3.5) is specified, not measured. No classifier is deployed against it and no client renders a capability manifest; nothing in §6 reflects it, and the claim made for it is only that it reuses the write path unchanged.

9 Conclusion

Availability under partition is a design ordering, not a feature. Putting it first forces specific and sometimes uncomfortable choices: identifiers derived from signed bytes rather than from the node that stores them, admission cost charged once at the origin rather than re-verified on receipt, canonical encoding used to admit rather than to verify, and a rejection where a more forgiving system would

repair. We have shown that this composition works from global reach down to bridged ISP islands, measured what it costs in latency, memory and bytes, and reported an availability defect that only a deployed multi-peer testbed could expose. What we have not done is run it during a real shutdown, under a real adversary, between operators who are strangers to each other. That is the measurement the design is actually asking for, and it is the work we think matters next.

References

[1] Martin R. Albrecht, Sofía Celi, Benjamin Dowling, and Daniel Jones. 2023. Practically-Exploitable Cryptographic Vulnerabilities in Matrix. In *2023 IEEE Symposium on Security and Privacy (SP)*. 164–181. doi:10.1109/SP46215.2023.10351027

[2] Russ Allbery and Charles Lindsey. 2009. *Netnews Architecture and Protocols*. Technical Report RFC 5537. Internet Engineering Task Force. doi:10.17487/RFC5537

[3] Ishaku Hassan Anaobi, Aravindh Raman, Ignacio Castro, Haris Bin Zia, Damilola Ibosiola, and Gareth Tyson. 2023. Will Admins Cope? Decentralized Moderation in the Fediverse. In *Proceedings of the ACM Web Conference 2023 (WWW)*. 3109–3120. doi:10.1145/3543507.3583487

[4] Leonhard Balduf, Saidu Sokoto, Onur Ascigil, Gareth Tyson, Björn Scheuermann, Maciej Korczyński, Ignacio Castro, and Michał Król. 2024. Looking AT the Blue Skies of Bluesky. In *Proceedings of the 2024 ACM on Internet Measurement Conference (IMC)*. 76–91. doi:10.1145/3646547.3688407

[5] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2012. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering* 2, 2 (2012), 77–89. doi:10.1007/s13389-012-0027-1

[6] Zachary S. Bischof, Kennedy Pitcher, Esteban Carisimo, Amanda Meng, Rafael Bezerra Nunes, Ramakrishna Padmanabhan, Margaret E. Roberts, Alex C. Snoeren, and Alberto Dainotti. 2023. Destination Unreachable: Characterizing Internet Outages and Shutdowns. In *Proceedings of the ACM SIGCOMM 2023 Conference*. doi:10.1145/3603269.3604883

[7] Bluesky Social. 2026. AT Protocol Repository Specification. https://atproto.com/specs/repository.

[8] Cecylia Bocovich, Arlo Breault, David Fifield, Serene, and Xiaokang Wang. 2024. Snowflake, a Censorship Circumvention System Using Temporary WebRTC Proxies. In *Proceedings of the 33rd USENIX Security Symposium*.

[9] David Chaum. 1983. Blind Signatures for Untraceable Payments. In *Advances in Cryptology — CRYPTO '82*. 199–203. doi:10.1007/978-1-4757-0602-4_18

[10] Ian Clarke, Oskar Sandberg, Brandon Wiley, and Theodore W. Hong. 2001. Freenet: A Distributed Anonymous Information Storage and Retrieval System. In *Designing Privacy Enhancing Technologies (Lecture Notes in Computer Science, Vol. 2009)*. 46–66. doi:10.1007/3-540-44702-4_4

[11] Cosmos SDK. 2020. ADR-020: Protocol Buffer Transaction Encoding. https://github.com/cosmos/cosmos-sdk/blob/main/docs/architecture/adr-020-protobuf-transaction-encoding.md.

[12] Rasmus Dahlberg and Tobias Pulls. 2018. Aggregation-Based Certificate Transparency Gossip. In *Proceedings of the 13th International Conference on Availability, Reliability and Security (ARES)*. doi:10.1145/3230833.3230869

[13] Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda. 2018. Privacy Pass: Bypassing Internet Challenges Anonymously. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2018. 164–180. doi:10.1515/popets-2018-0026

[14] Alex Davidson, Jana Iyengar, and Christopher A. Wood. 2024. *The Privacy Pass Architecture*. Technical Report RFC 9576. Internet Engineering Task Force. doi:10.17487/RFC9576

[15] Roger Dingledine and Nick Mathewson. 2006. *Design of a Blocking-Resistant Anonymity System*. Technical Report 2006-11-001. The Tor Project. https://svn.torproject.org/svn/projects/design-paper/blocking.pdf.

[16] Cynthia Dwork and Moni Naor. 1993. Pricing via Processing or Combatting Junk Mail. In *Advances in Cryptology — CRYPTO '92 (Lecture Notes in Computer Science, Vol. 740)*. 139–147. doi:10.1007/3-540-48071-4_10

[17] fiatjaf. 2023. NIP-01: Basic Protocol Flow Description. https://github.com/nostr-protocol/nips/blob/master/01.md.

[18] David Fifield, Chang Lan, Rod Hynes, Percy Wegmann, and Vern Paxson. 2015. Blocking-Resistant Communication Through Domain Fronting. *Proceedings on Privacy Enhancing Technologies* 2015, 2 (2015), 46–64. doi:10.1515/popets-2015-0009

[19] Alan Ford, Costin Raiciu, Mark Handley, Olivier Bonaventure, and Christoph Paasch. 2020. *TCP Extensions for Multipath Operation with Multiple Addresses*. Technical Report RFC 8684. Internet Engineering Task Force. doi:10.17487/RFC8684

[20] Google. 2024. Protobuf Serialization Is Not Canonical. https://protobuf.dev/programming-guides/serialization-not-canonical/.

[21] David Kaye. 2015. *Report of the Special Rapporteur on the Promotion and Protection of the Right to Freedom of Opinion and Expression*. Technical Report A/HRC/29/32. United Nations Human Rights Council. https://www.ohchr.org/en/documents/thematic-reports/ahrc2932-report-encryption-anonymity-and-human-rights-framework.

[22] Ari Keranen, Christer Holmberg, and Jonathan Rosenberg. 2018. *Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal*. Technical Report RFC 8445. Internet Engineering Task Force. doi:10.17487/RFC8445

[23] Ben Laurie, Adam Langley, and Emilia Kasper. 2013. *Certificate Transparency*. Technical Report RFC 6962. Internet Engineering Task Force. doi:10.17487/RFC6962

[24] Ben Laurie, Eran Messeri, and Rob Stradling. 2021. *Certificate Transparency Version 2.0*. Technical Report RFC 9162. Internet Engineering Task Force. doi:10.17487/RFC9162

[25] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology — CRYPTO '87*. 369–378. doi:10.1007/3-540-48184-2_32

[26] Nostr Protocol. 2026. NIP-77: Negentropy Syncing. https://github.com/nostr-protocol/nips/blob/master/77.md.

[27] Aravindh Raman, Sagar Joglekar, Emiliano De Cristofaro, Nishanth Sastry, and Gareth Tyson. 2019. Challenges in the Decentralised Web: The Mastodon Case. In *Proceedings of the Internet Measurement Conference (IMC)*. 217–229. doi:10.1145/3355369.3355572

[28] Tohidul Islam Raso, Suhadha Afrin, Miraj Ahmed Chowdhury, Maria Xynou, and Elizaveta Yachmeneva. 2025. The Longest Silence: Internet Shutdowns During Bangladesh's 2024 Uprising. Digitally Right and Open Observatory of Network Interference. https://ooni.org/post/2025-bangladesh-report/.

[29] Secure Scuttlebutt Consortium. 2026. Scuttlebutt Protocol Guide. https://ssbc.github.io/scuttlebutt-protocol-guide/.

[30] James M. Snell and Evan Prodromou. 2017. Activity Streams 2.0. W3C Recommendation. https://www.w3.org/TR/activitystreams-core/.

[31] Dominic Tarr, Erick Lavoie, Aljoscha Meyer, and Christian Tschudin. 2019. Secure Scuttlebutt: An Identity-Centric Protocol for Subjective and Decentralized Applications. In *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN)*. 1–11. doi:10.1145/3357150.3357396

[32] The Matrix.org Foundation. 2024. Matrix Specification, Version 1.11. https://spec.matrix.org/v1.11/.

[33] Marc Waldman and David Mazières. 2001. Tangler: A Censorship-Resistant Publishing System Based on Document Entanglements. In *Proceedings of the 8th ACM Conference on Computer and Communications Security (CCS)*. 126–135. doi:10.1145/501983.502002

[34] Marc Waldman, Aviel D. Rubin, and Lorrie Faith Cranor. 2000. Publius: A Robust, Tamper-Evident, Censorship-Resistant Web Publishing System. In *Proceedings of the 9th USENIX Security Symposium*. 59–72.

[35] Christopher Lemmer Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. 2018. ActivityPub. W3C Recommendation. https://www.w3.org/TR/activitypub/.

[36] Eric Wustrow, Scott Wolchok, Ian Goldberg, and J. Alex Halderman. 2011. Telex: Anticensorship in the Network Infrastructure. In *Proceedings of the 20th USENIX Security Symposium*.